

Deep learning for solving dynamic economic models

Lilia Maliar, Serguei Maliar and Pablo Winant

June 6

PKU-Zurich PhD Summer School on
Machine Learning for Macroeconomics and Finance
July 6–10, 2026 · Beijing, China

What is artificial intelligence (AI)

- AI is a device that:
 - has a goal (robot learns to walk)
 - perceives the environment (space around robot)
 - takes actions to achieve its goal (electrical impulses to rotors)
- AI look a lot like agent in economic models:
 - goal = return function (utility level)
 - environment = state variable (capital, productivity)
 - action = control (consumption)
- So, maybe, general-purpose AI technology can solve for us economic models without the need of writing model-specific codes?

Applications in economics and data science

- AI has many impressive applications:
 - recognition of handwriting and speech
 - composing Chopin music, playing Go
 - facilitating computer vision
- At the same time, economists cannot solve:
 - heterogeneous-agent and life-cycle models
 - dynamic games and IO models
 - expensive nonlinear estimation, etc.
- So, maybe machines can solve for us those difficult problems that we want but cannot currently solve?

Are economic models easy or hard for machines?

The answers the above question is not so clear

- It is actually unknown whether economic models are easy or hard for machines, they might be harder than operating self-driving cars
- We address this question in constructive way: we show how deep learning (DL) technology used in the modern data science can be applied to solving dynamic economic models
- Our DL framework builds on four main results that we discuss in the next four slides

Result 1: Casting economic models into expectation functions

- We offer a unified approach that makes it possible to cast 3 key objects of economic dynamics into expectation functions:
 - lifetime reward (indirect utility function)
 - Bellman equation
 - Euler equation
- In effect, we convert dynamic economic model into static optimization problem
- We solve the optimization problem by estimating a nonlinear regression using the deep learning tools

Result 2: Stochastic optimization for training

- We adapt stochastic gradient descent methods to training machines under the objective functions derived from economics
- In each iteration, we train machine on just one or few randomly chosen grid points
- Random grids replace conventional fixed grid with potentially large number of points
- We show that stochastic optimization can be also effectively combined with other approximating function like polynomials and splines (in addition to neural networks)

Result 3: All-in-one expectation operator

- We introduce a technique "all-in-one expectation operator" that merges two types of randomness:
 - current state variables (because our grid points are random)
 - future state variables (because shocks are random)
- We adapt the all-in-one operator with Kuhn-Tucker conditions via Fisher-Burmeister function to ensure that the stochastic gradient is unbiased
- One Monte Carlo draw is used to simultaneously implement both approximation and integration

Result 4: Google TensorFlow platform

- We show that it is possible to analyze economic models by using the same combination of software and hardware that lead to break-ground applications in data science:
- Ubiquitous python code designed to accommodate a variety of applications
- Model-free Google TensorFlow platform for simulation
- Our software is an open source

Numerical applications

We apply the DL framework to solve three models:

1. A consumption-saving problem with borrowing constraints via Fisher-Burmeister function
2. Krusell and Smith (1998) model with 1000 agents - Krusell and Smith (1998) replaced the distribution with its moments, reducing the state space to 4 or 5 variables
 - In turn, we construct decision functions of thousands of variables: we literally put the entire distribution into neural networks
3. Large new Keynesian model of the Bank of Canada

Model reduction

- How can we deal with so huge state space?
 - Neural network automatically performs model reduction!
- *It learn to extract information from many inputs and condense it into a smaller set of hidden layers: this is how we deal with thousands of state variables*
- *It learns to ignore redundant and collinear variables: this is how we handle ill conditioning*

Relation to the computational economics

- The lifetime-reward maximization method is related to an indirect inference procedure of Smith (1987).
- The Euler-equation method is related to seminal contributions to numerical solution methods in economics: the projection method of Judd (1992) and PEA of Den Haan and Marcet (1990).
- The Bellman-equation method is related to conventional value and policy function iteration (see e.g., Rust, 1996, Judd, 1998, Santos, 1999, Aruoba et al., 2006, Stachurski, 2009).

Relation to the learning literature

Our method is related to three fields.

- Supervised Learning: Duffy and McNelis (2001), Duarte (2018), Fernandez-Villaverde et al. (2018), Villa and Valaitis (2019), Lepetyuk et al. (2019), Azinovic et al. (2019)
- Unsupervised Learning: Judd, Maliar and Maliar (2011), Maliar and Maliar (2015), Judd et al. (2017), Coleman et al.(2018), Villa and Valaitis (2019)
- Reinforcement Learning: Sutton and Barto (2018), Powell (2010), Lepetyuk and Jirniy (2012)

A deep learning algorithm for optimization

1. Define theoretical and empirical loss function
 $\Xi(\theta) = E_{\omega}[\xi(\omega; \theta)]$, and $\Xi_n(\theta) = \frac{1}{n} \sum_{i=1}^n \xi(\omega_i; \theta)$ and $\{\omega_i\}_{i=1}^n$ is the given set of features (data points).
2. Define a family of parametric functions $\{\varphi(\cdot; \theta) : \theta \in \mathbb{R}^{d-\theta}\}$, where θ = parameters vector.
3. Train the machine, i.e., find θ that minimizes the empirical loss, typically using stochastic gradient or batch stochastic gradient methods.
4. Check accuracy of the constructed approximation.

We are just left to frame the economic model as an objective for the AI minimization!

Stochastic Gradient

- Gradient Descent: (learning rate λ)

$$\theta_{k+1} \leftarrow \theta_k - \lambda_k \nabla_{\theta} E_{\omega} [\xi(\omega; \theta)]$$

- Stochastic Gradient Descent:

$$\theta_{k+1} \leftarrow \theta_k - \lambda_k \nabla_{\theta} \xi(\omega; \theta)$$

- Batch Stochastic Gradient Descent*:

$$\theta_{k+1} \leftarrow \theta_k - \lambda_k \sum_{k=1}^N \nabla_{\theta} \xi(\omega_k; \theta)$$

How will we apply this framework to economics?

We show three objectives for a growth model:

- lifetime utility of a consumer (to be maximized)
- residuals in Bellman equation (to be minimized)
- residuals in Euler equation (to be minimized)

TensorFlow

TensorFlow - an open source software library for Deep Learning developed by Google in 2011 and subsequently released in November 2015

- "TensorFlow" = "Tensor" + "Flow"
- "Tensor": multidimensional vectors used to represent the data (tensors as in math)
- "Flow": the flow through which the tensors cross the network

TensorFlow (cont.)

Extremely efficient:

- automatic differentiation
- parallel computation is cheap (GPUs, clusters)
- postpones the execution of the operations to the last possible moment
- avoids carrying out the same operation twice
- interfaces with the Python libraries
- implementation is in Python but computations are in C ++;
- easy debugging with TensorBoard.

Casting dynamic economic models into expectation functions

A class of dynamic economic models

- We consider dynamic Markov economic models with time-invariant decision functions
- Optimization problem: An exogenous state $z_{t+1} \in \mathbb{R}^{n_z}$ follows a Markov process driven by an i.i.d. innovation process $\epsilon_t \in \mathbb{R}^z$ with a transition function Z ,

$$z_{t+1} = Z(z_t, \epsilon_t)$$

Casting dynamic economic models into expectation functions

A class of dynamic economic models (cont.)

- An endogenous state x_{t+1} is driven by

$$x_{t+1} = X(z_t, x_t, y_t, z_{t+1})$$

- z_t = exogenous state, $y_t \in \mathbb{R}^{n_y}$ = choice, X = transition function.
 - The choice y_t satisfies the constraint:

$$y_t \in Y(z_t, x_t).$$

- where (z_t, x_t) and y_t determine the period reward $r(z_t, x_t, y_t)$.

Optimization problem

- The agent maximizes discounted lifetime reward:

$$\max_{\{y_t, x_{t+1}\}_{t=0}^{\infty}} E_0 \sum_{t=0}^{\infty} \beta^t r(z_t, x_t, y_t)$$

- $\beta \in [0, 1)$ = discount factor, $E_0 [\cdot]$ = expectation function across future shocks $(\epsilon_1, \epsilon_2, \dots)$ conditional on (z_0, x_0)

Decision rule

- A decision rule is a function $\varphi : \mathbb{R}^{n_z} \times \mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_y}$ such that $y_t = \varphi(z_t, x_t) \in Y(z_t, x_t)$.
- An approximate decision rule is a member of a family of functions $\varphi(\cdot; \theta)$ parameterized by a real vector θ such that $\varphi(z_t, x_t; \theta) \in Y(z_t, x_t)$.

Objective 1: Lifetime-reward maximization

Two types of randomness in $E_0[\cdot]$:

- a random state (z_0, x_0)
- a random future shocks $(\epsilon_1, \dots, \epsilon_T)$.

All-in-one expectation operator:

- Fix time horizon $T > 0$, select a decision rule $\varphi(\cdot; \theta)$, and define the distribution of $\omega \equiv (z_0, x_0, \epsilon_1, \dots, \epsilon_T)$.
- For given θ , lifetime reward associated with the rule $\varphi(\cdot; \theta)$ is given by

$$\Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] = E_{(z_0, x_0, \epsilon_1, \dots, \epsilon_T)} \sum_{t=0}^T \beta^t r(z_t, x_t, \varphi((z_t, x_t; \theta)))$$

Objective 1: Lifetime-reward maximization (cont.)

- A remarkable distributive property of

$$\Xi(\theta) = E_{(z_0, x_0, \epsilon_1, \dots, \epsilon_T)} \sum_{t=0}^T \beta^t r(z_t, x_t, \varphi((z_t, x_t; \theta)))$$

- A single random draw or batch is used to evaluate all the expectation functions and stochastic gradients in the model at once.
- With n random draws, an integration method that constructs the two expectation functions separately would require n draws for evaluating expectation w.r.t. (z_0, x_0) and n' draws for evaluating the expectation function w.r.t. $(\epsilon_1, \dots, \epsilon_T)$.
- $n \times n'$ draws in total.

Objective 2: Euler-residual minimization

Euler equations: $E_\epsilon[f_j(z, x, y, z', x', y')] = 0, \quad j = 1, \dots, J$

- Select a decision rule $\varphi(\cdot; \theta)$, and define the distribution of random variables $\omega \equiv (z, x)$.
- For given θ , the squared residuals in the Euler equations:

$$\Xi(\theta) = E_\omega[\xi(\omega; \theta)] = E_{(z, x)}\left[\sum_{j=1}^J v_j (E_\epsilon[f_j(z, x, y, z'(\epsilon), x'(\epsilon), y'(\epsilon))])^2\right],$$

$(v_1, \dots, v_J)$ = vector of weights on J optimality conditions.

- 2 expectations $E_{(z, x)}[\cdot]$ and $E_\epsilon[\cdot]$.

Objective 3: Bellman-residual minimization

- Bellman equation:

$$V(z, x) = \max_{y, x'} \{r(z, x, y) + \beta E_{\epsilon}[V(z'(\epsilon), x'(\epsilon))]\}$$

s.t. $z' = z(z, \epsilon)$, $x' = X(z, x, y, z')$ and $y \in Y(z, x)$.

- Bellman-residual minimization:

$$\min_{z, x} E_{(z, x)} [V(z, x) - \max_{y, x'} \{r(z, x, y) + \beta E_{\epsilon}[V(z'(\epsilon), x'(\epsilon))]\}]^2$$

Objective 3: Bellman-residual minimization (cont.)

- 3 approaches to constructing a solution

i) direct optimization (grid search, numerical solver);

ii) FOC;

iii) the envelope conditions (EC).

- One can formulate them in terms of the objective functions to minimize:

$$\begin{aligned}\zeta^{\max} &= -\{r(z, x, y) + \beta E_{\epsilon}[V(z'(\epsilon), x'(\epsilon); \theta_1)]\}, \\ \zeta^{FOC} &= \{r_2(z, x, y) - \beta E_{\epsilon}[V_2(z'(\epsilon), x'(\epsilon); \theta_1)]\}^2, \\ \zeta^{EC} &= \{V_2(z, x; \theta_1) - r_2(z, x, y)\}^2\end{aligned}$$

- Traditional value iterative methods can be expensive.

Objective 3: Bellman-residual minimization (cont.)

- Our approach: we combine a minimization of the residuals in the Bellman equation with a minimization of $\zeta \in \{\zeta^{\max}, \zeta^{FOC}, \zeta^{EC}\}$.
- Select a value function $V(\cdot; \theta_1)$ and decision rule $y = \varphi(\cdot; \theta_2)$, define a distribution of $\omega \equiv (z, x)$.
- For given $\theta \equiv (\theta_1, \theta_2)$, the squared residuals in the Bellman equations:

$$\begin{aligned}\Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] = \\ E_{(z,x)}\{V(z, x; \theta_1) - r(z, x, y) - \beta E_{\epsilon}[V(z'(\epsilon), x'(\epsilon); \theta_1)]\}^2 \\ + v E_{(z,x)}\{\zeta(z, x, \epsilon; \theta)\}\end{aligned}$$

All-in-one expectation operators

- In lifetime-reward maximization, expectation operators are related linearly and can be combined into one

$$E_{(z_0, x_0)}[E_{(\epsilon_1, \dots, \epsilon_T)} r(\cdot)] = E_{(z_0, x_0, \epsilon_1, \dots, \epsilon_T)}[r(\cdot)]$$

- In Euler- and Bellman-residual minimization, expectation operator E_ϵ is squared and cannot be naturally combined.

$$E_{(z, x)}(E_\epsilon[f_j(z, x, \epsilon)])^2 \neq E_{zx} E_\epsilon[f_j(z, x, \epsilon)^2]$$

- As a result, stochastic gradient is biased

$$E_{(z, x)}(E_\epsilon[\nabla f_j(z, x, \epsilon)])^2 \neq \nabla E_{(z, x)}(E_\epsilon[f_j(z, x, \epsilon)])^2$$

Our uncorrelated shocks method

- We offer a technique that allows us to combine the expectation functions $E_{zx}[\cdot]$ and $E_{\epsilon}[\cdot]$ in a single expectation operator in the presence of squares.
- We use two independent random draws or two batches ϵ_1 and ϵ_2 for the two terms:

$$E_{\epsilon_1}[f(\epsilon_1)]E_{\epsilon_2}[f(\epsilon_2)] = E_{(\epsilon_1, \epsilon_2)}[f(\epsilon_1)f(\epsilon_2)]$$

- As a result, the stochastic gradient descent method is unbiased.

All-in-one expectation operators for Euler equations

- Select a decision rule $\varphi(\cdot; \theta)$, define a distribution of random variable $\omega = (z, x, \epsilon_1, \epsilon_2)$.
- For a given θ , the squared residuals:

$$\begin{aligned}\Xi(\theta) &= E_{\omega}[\xi(\omega; \theta)] = E_{(z, x, \epsilon_1, \epsilon_2)} \simeq \\ &\sum_{j=1}^J v_j f_j(z, x, y, z'(\epsilon_1), x'(\epsilon_1), y'(\epsilon_1)) f_j(z, x, y, z'(\epsilon_2), x'(\epsilon_2), y'(\epsilon_2))\end{aligned}$$

$(v_1, \dots, v_J)$ = vector of weights on J FOCs.

All-in-one expectation operators for Bellman equations

- Select value function $V(\cdot; \theta_1)$ and decision rule $y = \varphi(\cdot; \theta_2)$, and define the distribution of the random variable $\omega = (z, x)$.
- For given $\theta = (\theta_1, \theta_2)$, the squared residuals:

$$\begin{aligned}\Xi(\theta) = E_{\omega} [\xi(\omega; \theta)] = \\ E_{(z, x, \epsilon_1, \epsilon_2)} \{ [V(z, x; \theta_1) - r(z, x, y) - \beta V(z'(\epsilon_1), x'(\epsilon_1); \theta_1)] \\ + v[r_2(z, x, y) - \beta E_{\epsilon} [V_2(z'(\epsilon_1), x'(\epsilon_1); \theta_1)] \\ \times [r_2(z, x, y) - \beta E_{\epsilon} [V_2(z'(\epsilon_1), x'(\epsilon_1); \theta_1)]] \end{aligned}$$

$v > 0$ = exogenous relative weight of the two objectives.

Consumption-saving problem

Consumption-saving problem with multiple shocks

$$\max_{c_t, w_t} E_0 \left[\sum_{t=0}^{\infty} \beta^t e^{\chi_t} u(c_t) \right]$$

subject to

$$\begin{aligned} w_{t+1} &= r e^{\varrho_t} (w_t - c_t) + e^{y_t} e^{p_t(1-\mu)} \\ c_t &\leq w_t, \\ (z, w) &\text{ is given.} \end{aligned}$$

where c_t = consumption; w_t = cash-on-hand;

$\{y_t, p_t, \varrho_t, \chi_t\} \equiv z_t$ = exog. shocks: temporary income y_t , permanent income p_t , interest-rate ϱ_t and preference χ_t shocks; $\mu \in \{0, 1\}$ = indicator function; $r \in (0, \frac{1}{\beta})$ = constant interest rate.

Consumption-saving problem

- Each $z \in \{y, p, \varrho, \chi\}$ follows an AR(1) process:
 $z_{j,t+1} = \rho_j z_{j,t} + \sigma_j \epsilon_{j,t+1}$ and $\epsilon_{j,t+1} \sim \mathcal{N}(0, 1)$, $|\rho_j| < 1$ and $\sigma_j > 0$.
- Bellman equation: $V(z, w) = \max_{c, w'} \{u(c) + \beta E_\epsilon [V(z', w')]\}$
- Kuhn-Tucker conditions: $c - w \leq 0, h \geq 0$ and $(c - w)h = 0$,
- $h \equiv u'(c)e^{x-\varrho} - \beta r E[u'(c')e^{x'}] = \text{Lagrange multiplier.}$

Calibration

- $u(c) = \frac{c^{1-\gamma}-1}{1-\gamma}$, $\gamma = 2$,
- $\beta = 0.9$, $r = 1.04$, $\sigma_y = 0.1$.
- Baseline uni-shock model: a temporary i.i.d. income shock y .
- Multi-shock model: $\rho_y = 0.9$ and $\sigma_y = 0.1$; $\rho_p = 0.999$ and $\sigma_p = 0.001$; $\rho_\varrho = 0.2$ and $\sigma_\varrho = 0.001$; $\rho_\chi = 0.9$ and $\sigma_\chi = 0.01$.

Computational details

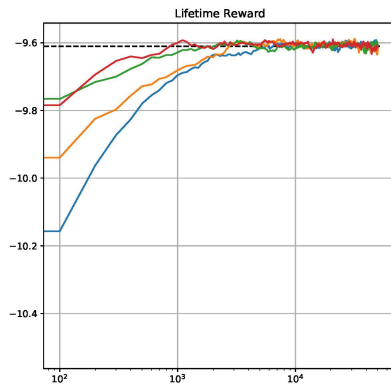
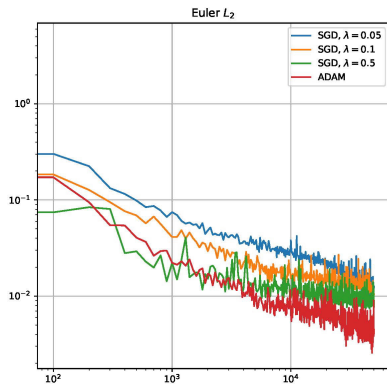
- Our code is in Python and Google's TensorFlow.
- Decision function:
 - parameterize $\frac{c_t}{w_t} \equiv \zeta_t = \sigma(\zeta_0 + \varphi(z_t, w_t; \theta))$, where $\varphi(\cdot)$ = neural network with 2 hidden layers,
 - $\sigma(x) = \frac{1}{1+e^{-x}}$ = sigmoid function, - each of the hidden layers includes 32 (leaky) relu neurons.
 - Value function: - Parameterize $V_t = V_0 + \varphi(z_t, w_t; \theta)$.

Computational details (cont.)

- Training: 16 random draws
- Fix $T > 0$, select $\varphi(\cdot; \theta)$, define a random vector $\omega \equiv (z_0, w_0, \epsilon_1, \dots, \epsilon_T)$ composed of an initial state and a realization of exogenous shocks.
- For a given random draw ω ,

$$\Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] \equiv E_{\omega}\left[\sum_{t=0}^T \beta^t u(c(z_t, w_t; \theta))\right]$$

Objective 1: Lifetime reward maximization (cont.)



Objective 2: Euler-equation method with Kuhn-Tucker conditions

- Fischer-Burmeister (FB) function:

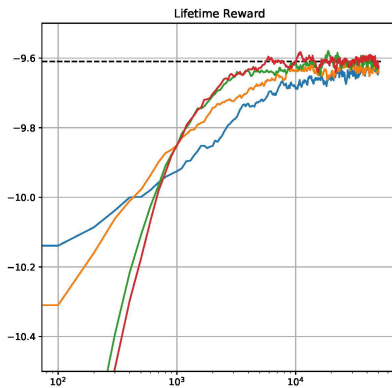
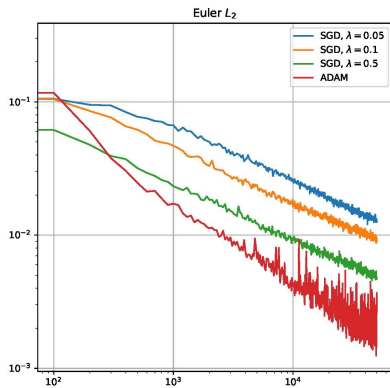
$$\Psi^{FB}(a, b) = a + b - \sqrt{a^2 + b^2} = 0$$

with $a = c - w$ and $b = c - u'^{-1}(\beta re^{\varrho t} E_{\epsilon}[u'(c')])$

- solution: $a \geq 0$, $b \geq 0$ and $ab = 0$
- similar to $\Psi^{\min}(a, b) = \min\{a, b\} = 0$ but differentiable
- Objective function: Select $\varphi(\cdot; \theta)$, draw $\omega \equiv (z, w)$,

$$\Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] \equiv E_{\omega}[\Psi^{FB}(c - w, c - u'^{-1}(\beta re^{\varrho t} E_{\epsilon}[u'(c')]))]^2$$

Objective 2:



Objective 3: Bellman-residual minimization

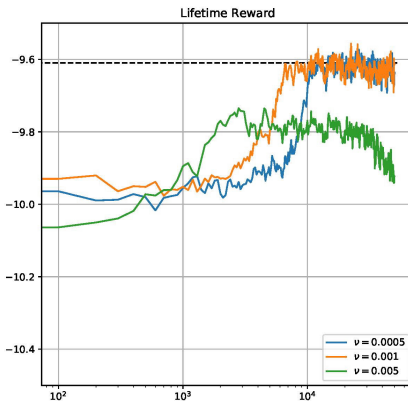
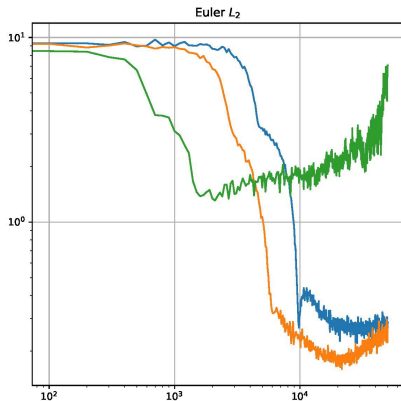
- We focus on the objective function which does not assume differentiability:

$$\zeta^{\max} = -\{r_2(z, x, y) + \beta E_{\epsilon}[V_2(z'(\epsilon), x'(\epsilon); \theta_1)]\}$$

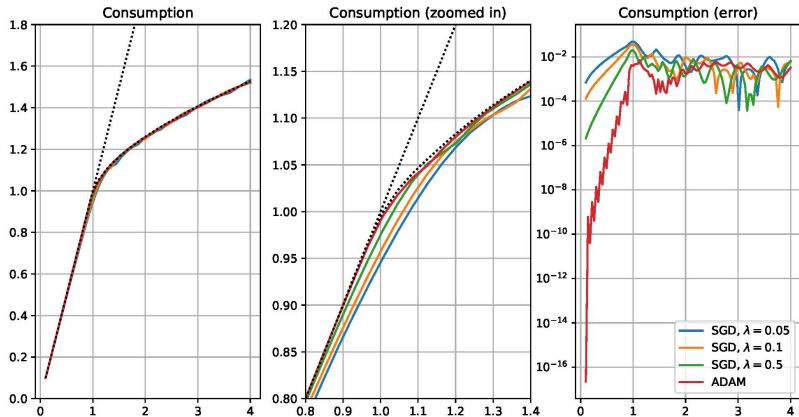
- Select a value function $V(\cdot; \theta_1)$ and a consumption rule $\varphi(\cdot; \theta_2)$, draw random state (z, w) and define residuals by

$$\begin{aligned}\Xi(\theta) &= E_{\omega}[\xi(\omega; \theta)] = \\ &= E_{(z, w)}[V(z, w; \theta_1) - u(c) - \beta E_{\epsilon}V(z', w'; \theta_1)]^2 \\ &\quad - vE_{(z, w)}[u_1(c) + \beta E_{\epsilon}V_2(z', w'; \theta_1)]\end{aligned}$$

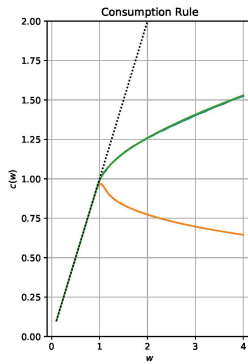
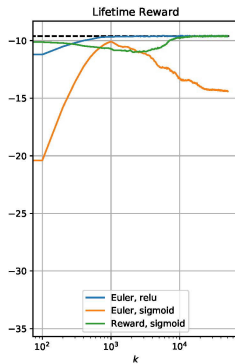
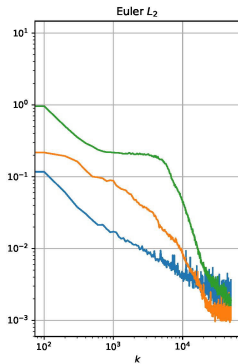
Objective 3: Bellman-residual minimization



Consumption decision rule



Role of activation functions



Scale-up

- The multi-shock model: 4 AR(1) shocks

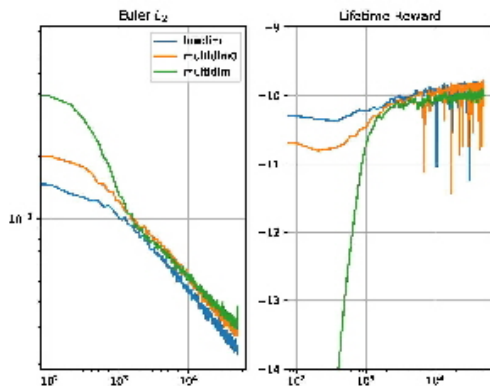
$\{y_t, p_t, \varrho_t, \chi_t\} \equiv z_t - z_{j,t+1} = \rho_j z_{j,t} + \sigma_j \epsilon_{j,t+1}$ and $\epsilon_{j,t+1} \sim \mathcal{N}(0, 1)$
recall: $\rho_y = 0.9$ and $\sigma_y = 0.1$; $\rho_p = 0.999$ and $\sigma_p = 0.001$; $\rho_\varrho = 0.2$ and $\sigma_\varrho = 0.001$; $\rho_\chi = 0.9$ and $\sigma_\chi = 0.01$

- Does dimension reduction work?
- Compare learning with the full model and with $\{p_t, \varrho_t, \chi_t\}$ muted: useless states should be eliminated

Scale-up: 3 cases considered

- "multidim": the model with 4 shocks
- "lowdim" and "multidim0": 1 shock, y_t , with $\rho_y = 0.9$ but differ in inputs supplied to the neural network
- "lowdim": $\{p_t, \varrho_t, \chi_t\}$ are not supplied at all - "multidim0": $\{p_t, \varrho_t, \chi_t\}$ are supplied but set equal to zero $\Rightarrow$ perfect multicollinearity

Dealing with multicollinearity



Training with the reward-maximization method

Yes, AI scales great!

- training properties don't depend on number of shocks (computational time)
- scales up *linearly*:
- when replace $\{w_t, y_t\}$ by $\{w_t, y_t, p_t, \varrho_t, \chi_t\}$, only $\theta^{(1)}$ changes

The multishock model

- Neural networks can
- deal with multicollinearity;
- deal with ill conditioning;
- do the model reduction.
- Convergence rate in the 4-shock model is the same as that of the other models.

⇒ The proposed solution method is potentially tractable in problems with high dimensionality.

All all-in-one expectation operator

- Let μ = supplementary variable representing the expectation function, v = exogenous weight

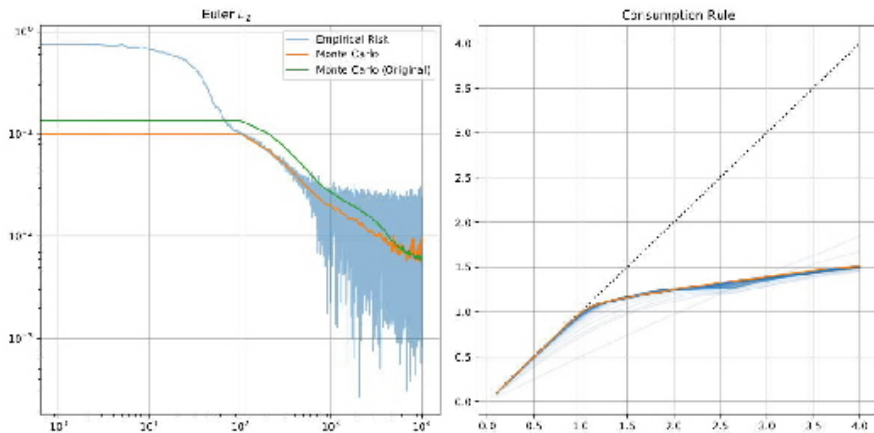
$$-E_{(z,w)}[\Psi(c-w, u'(c) - \mu)]^2 + v[\beta re^{qt} E_{\epsilon}[u'(c'(\epsilon))] - \mu]^2$$

- Euler-residual minimization - combine integration w.r.t. z , w and ϵ :

$$\begin{aligned} E_{\omega}[\xi(\omega; \theta)] &= E_{(z,w,\epsilon_1,\epsilon_2)} = \\ &\quad \{[\Psi(c-w, u'(c) - \mu)]^2 \\ &\quad + v[\beta re^{qt} E_{\epsilon}[u'(c'(\epsilon_1))] - \mu][\beta re^{qt} E_{\epsilon}[u'(c'(\epsilon_2))] - \mu]\} \end{aligned}$$

- Bellman-residual minimization - directly applicable (with a change of notation).

Training with Euler-residual method in the 4-shock model



Batch size varies from $N = 8$ to $N = 8192$ draws

Krusell and Smith (1998) model

Can AI work in high-dimensional problems?

- neural networks:
- flexible functional form (kinks)
- robust to collinearity
- implicit dimensionality reduction
- generic training method
- scales up well
- lots of hyperparameters to be tuned:**
- learning method (SGD, ADAM, AdaGrad)
- neural network topology (activation functions, width)

Heterogeneous-agent set up

Agents $i = 1, \dots, \ell$. Identical in fundamentals but differ in productivity and capital.

$$\max_{c_t^i, k_t^i} E_0 \left[\sum_{t=0}^{\infty} \beta^t u(c_t^i) \right]$$

s.t.

$$\begin{aligned} c_t^i + k_{t+1}^i &= R_t k_t^i + W_t z_t^i, \\ z_{t+1}^i &= \rho_z z_t^i + \sigma_z \epsilon_t^i, \text{ with } \epsilon_t^i \sim \mathcal{N}(0, 1) \\ k_{t+1}^i &\geq 0, \text{ and } (k_0^i, z_0^i) \text{ is given} \end{aligned}$$

c_t^i, k_t^i are z_t^i = consumption, capital, labor productivity

Heterogeneous-agent set up (cont.)

- Cobb-Douglas production function $z_t k_t^\alpha$, with $k_t = \sum k_t^i = \text{agg. capital}$, $z_t = \text{agg. productivity}$.

$$\begin{aligned} R_t &= 1 - d + z_t \alpha k_t^{\alpha-1} \\ z_{t+1} &= \rho z_t + \sigma \epsilon_t, \epsilon_t \sim \mathcal{N}(0, 1) \end{aligned}$$

- State space: $\{k_t^i, z_t^i, \{k_t^i, z_t^i\}_{i=1}^I, z_t\}$
- Krusell and Smith's (1998) method: replace distributions with a finite set of moments $z_t \Rightarrow$ approximate state space by $\{k_t^i, z_t^i, k_t, z_t\}$.

Deep learning solution algorithm

- We work directly with the actual state space.
- Objective function:

$$E_{\omega}[\xi(\omega; \theta)] = E_{(K, Z, z, \Sigma_1, \Sigma_2, \epsilon_1, \epsilon_2)} \{ [\Psi(c^i - w^i, u'(c^i) - \mu^i)]^2 + \\ v[\beta E_{\Sigma_1, \epsilon_1} [u'(c^{i'}(\Sigma_1, \epsilon_1))] - \mu^i] \times \\ [\beta E_{\Sigma_2, \epsilon_2} [u'(c^{i'}(\Sigma_2, \epsilon_2))] - \mu^i] \}$$

where $K = (k_1^1, \dots, k_1^\ell)$, $Z = (z_1^1, \dots, z_1^\ell)$, $\Sigma_1 = (\epsilon_1^1, \dots, \epsilon_1^\ell)$,
 $\Sigma_2 = (\epsilon_2^1, \dots, \epsilon_2^\ell)$; $\epsilon_1, \epsilon_2 = \text{agg. productivity shocks}$.

Deep learning solution algorithm (cont.)

Procedure

- i) draw initial state z_0
- ii) compute aggregate capital k_0 and prices R_0, W_0 ;
- iii) train neural network for ℓ agents;
- iv) compute next period distribution z_1 and $\{K_1, Z_1\} = \{k_1^i, z_1^i\}_{i=1}^\ell$.

Proceed iteratively until convergence.

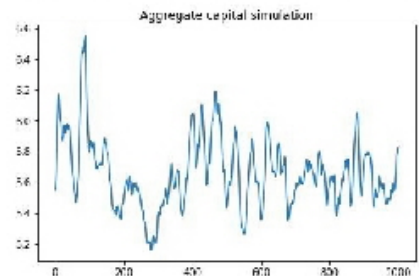
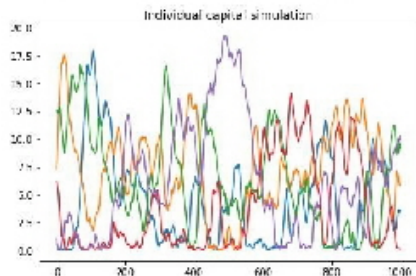
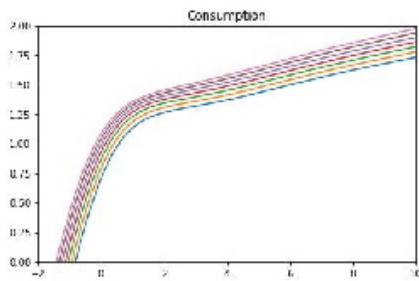
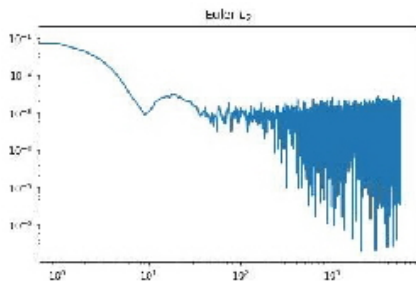
Deep learning solution algorithm (cont.)

- Ergodic-set domain - produce the state variables by simulating the economy forward.
- Perfect multicollinearity - include the state variables of agent i twice, $\varphi^i = \varphi(k_t^i, z_t^i, \{K_t, Z_t\}, z_t; \theta)$
- Model reduction - solve the models with up to $\ell = 1,000$ of agents ($2\ell + 1 = 2,001$ state variables).

Calibration and computational details

- $u(c) = \frac{c^{1-\gamma}-1}{1-\gamma}$ with $\gamma = 1 - \beta = 0.96$; $\rho = 0.9$; $\sigma = 0.07$; $\rho_z = 0.9$; $\sigma_z = 0.4(1 - \rho_z^2)^{1/2}$.
- ADAM with batch size of 10 and learning rate of 0.001.
- $K = 100,000$ iterations (equal to simulation length).

Training with Euler-residual method, 5 agents



Conclusion: Enthusiastic view

- Data scientists developed powerful AI techniques for data analysis (e.g., deep learning)
- created software for their efficient implementation (e.g., TensorFlow)
- Economists mostly develop own method - not portable

Grain of salt, black magic and no-free-lunch

- We offer AI-based technology but do not know whether it is best possible for all economic models.
- Several reasons to doubt it:
- Neural network are promising and universal approximators but have a large number of parameters and are highly non-linear.
- Convergence to global solution is not guaranteed.
- Solutions in economic models are often well described by simple polynomials.
- So, it is not clear if neural network is useful or not.

Grain of salt, black magic and no-free-lunch

- Second, Monte Carlo integration lying in the basis of AI technology is simple but has a low, square-root rate of convergence.
- Mindless drawing of random number is not necessarily the best.
- More efficient outcomes can be obtained if we engineer better sequences or grids to enhance interpolation or integration results.

Grain of salt, black magic and no-free-lunch

- Third, the stochastic optimization is magical: it replaces numerical approximation of integral over a large number of points with an evaluation of its integrand in few of such points.
- But there is no free lunch: its convergence rate is slower and not guaranteed.
- The latter method is more expensive than stochastic gradient and may be intractable for complex approximating functions.
- Finally, there is a variety of AI technologies that can be used for solving dynamic economic models.

Grain of salt, black magic and no-free-lunch

- Alphazero plays Go but not chess and Deepblue plays chess but not Go.
- Again, there is no free lunch: for one application where a method works, there is another where it does not.
- The idea that there is an all-purpose procedure, which will remove the curse of dimensionality, without any trade-offs, for any kind of models, and will produce highly precise and uniformly accurate solution belongs to magical thinking.
- The human input is still critical for understanding and exploiting the trade-offs

Grain of salt, black magic and no-free-lunch

- The bottom line is that we probably should not retire all computational economists at the moment.
- At least, not until they figure out what type of AI works best for economic models.